

BSc (Hons) Computing Systems

COM661 (18368) - Full-stack Strategies and Development

Code File

Author

B00957996 : Md Nasif

app.py

```
from flask import Flask, make_response

from auth.auth import auth_bp
from employees.employees import employee_bp
from departments.departments import department_bp
from roles.roles import role_bp
from attendance.attendance import attendance_bp
from leaves.leaves import leave_bp
from admin.admin import admin_bp
from analytics.analytics import analytics_bp

app = Flask(__name__)

# Register all Blueprints
app.register_blueprint(auth_bp)
app.register_blueprint(employee_bp)
app.register_blueprint(department_bp)
app.register_blueprint(role_bp)
app.register_blueprint(attendance_bp)
app.register_blueprint(leave_bp)
app.register_blueprint(admin_bp)
```

```
app.register_blueprint(analytics_bp)


# Welcome route
@app.route("/", methods=["GET"])
def index():

    return make_response(
        "<h1>Welcome to Employee Management</h1>",
        200
    )

if __name__ == "__main__":
    app.run(debug=True)
```

config.py

```
SECRET_KEY = 'topsecretkey'
```

database.py

```
from pymongo import MongoClient

# MongoDB connection
client = MongoClient("mongodb://localhost:27017")

# Database
db = client.em_management

# Collections
employees = db.employees
departments = db.departments
roles = db.roles
attendance = db.attendance
leave_requests = db.leave_requests
users = db.users
blacklist_token = db.blacklist_token
```

decorators.py

```
from flask import request, jsonify, make_response
from functools import wraps
import jwt

from database import blacklist_token
from config import SECRET_KEY


# Admin Wrapper
def admin_required(func):

    @wraps(func)
    def admin_required_wrapper(*args, **kwargs):

        token = request.headers.get('x-access-token')

        if not token:
            return make_response(jsonify({
                'Message': 'Token is missing'
            }), 401)
```

```
        try:
            data = jwt.decode(
                token,
                SECRET_KEY,
                algorithms=['HS256']
            )

        except jwt.InvalidTokenError:
            return make_response(jsonify({
                'Message': 'Token is invalid'
            }), 401)

        if data['admin']:
            return func(*args, **kwargs)

        else:
            return make_response(jsonify({
                'Message': 'Admin access required!'
            }), 401)

    return admin_required_wrapper

# JWT Wrapper
```

```
def jwt_required(func):

    @wraps(func)
    def jwt_required_wrapper(*args, **kwargs):

        token = None

        if 'x-access-token' in request.headers:
            token = request.headers['x-access-token']

        if not token:
            return make_response(jsonify({
                'Message': 'Token is missing'
            }), 401)

        try:
            data = jwt.decode(
                token,
                SECRET_KEY,
                algorithms=['HS256']
            )

        except jwt.InvalidTokenError:
```

```
            return make_response(jsonify({
                'Message': 'Token is invalid'
            }), 401)

        bl_token = blacklist_token.find_one({
            'token': token
        })

        if bl_token is not None:
            return make_response(jsonify({
                'Message': 'Token has been cancelled!'
            }), 401)

        return func(*args, **kwargs)

    return jwt_required_wrapper
```

admin.py

```
from flask import Blueprint, request, jsonify, make_response

from datetime import datetime
```

```
from database import (
    users,
    employees,
    departments,
    roles,
    attendance,
    leave_requests
)

from decorators import jwt_required, admin_required

admin_bp = Blueprint('admin', __name__)

#route to get all users

@admin_bp.route('/api/v1.0/admin/users', methods=['GET'])
@jwt_required
@admin_required
def get_all_users():
    query = {}

    if request.args.get('username'):
        query['username'] = {
```

```
        '$regex': request.args.get('username').strip(),
        '$options': 'i'
    }

if request.args.get('email'):
    query['email'] = {
        '$regex': request.args.get('email').strip(),
        '$options': 'i'
    }

if request.args.get('role'):
    role = request.args.get('role').strip()

    if role not in ['admin', 'hr_staff', 'manager','employee']:
        return make_response(jsonify({
            'Error': 'Invalid user role.'
        }), 400)

    query['role'] = role

data_to_return = []

for user in users.find(query, {'password': 0}).sort('user_id', 1):
```

```
        user['_id'] = str(user['_id'])
        data_to_return.append(user)

    return make_response(jsonify(data_to_return), 200)

#route to get one user

@admin_bp.route('/api/v1.0/admin/users/<string:user_id>',
methods=['GET'])
@jwt_required
@admin_required
def get_one_user(user_id):
    user = users.find_one(
        {'user_id': user_id},
        {'password': 0}
    )

    if user is None:
        return make_response(jsonify({
            'Error': 'User not found.'
        }), 404)

    user['_id'] = str(user['_id'])
```

```
    return make_response(jsonify(user), 200)


#route to update an user


@admin_bp.route('/api/v1.0/admin/users/<string:user_id>',
methods=['PUT'])
@jwt_required
@admin_required
def update_user(user_id):
    user = users.find_one({'user_id': user_id})

    if user is None:
        return make_response(jsonify({
            'Error': 'User not found.'
        }), 404)

    data = request.form.to_dict()

    if not data:
        return make_response(jsonify({
            'Error': 'No form data was provided.'
        }), 400)

    allowed_fields = ['username', 'email', 'role']
```

```
invalid_fields = [
    field for field in data
    if field not in allowed_fields
]

if invalid_fields:
    return make_response(jsonify({
        'Error': 'One or more fields cannot be updated.',
        'Invalid fields': invalid_fields
    }), 400)

for field in data:
    data[field] = data[field].strip()

    if not data[field]:
        return make_response(jsonify({
            'Error': f'{field} cannot be empty.'
        }), 400)

if 'role' in data:
    if data['role'] not in ['admin', 'hr_staff', 'manager','employee']:
        return make_response(jsonify({
```

```
            'Error': 'Invalid user role.'
        }), 400)

    if 'username' in data:
        existing_username = users.find_one({
            'username': data['username'],
            'user_id': {'$ne': user_id}
        })

        if existing_username:
            return make_response(jsonify({
                'Error': 'This username already exists.'
            }), 409)

    # Check for an existing email
    if 'email' in data:
        existing_email = users.find_one({
            'email': data['email'],
            'user_id': {'$ne': user_id}
        })

        if existing_email:
            return make_response(jsonify({
```

```
            'Error': 'This email already exists.'
        }), 409)

    data['updated_at'] = datetime.now().isoformat(timespec='seconds')

    users.update_one(
        {'user_id': user_id},
        {'$set': data}
    )

    updated_user = users.find_one(
        {'user_id': user_id},
        {'password': 0}
    )

    updated_user['_id'] = str(updated_user['_id'])

    return make_response(jsonify({
        'Message': 'User updated successfully.',
        'User': updated_user
    }), 200)

#route to delete a user
```

```
@admin_bp.route('/api/v1.0/admin/users/<string:user_id>',
methods=['DELETE'])
@jwt_required
@admin_required
def delete_user(user_id):
    user = users.find_one({
        'user_id': user_id
    })

    if user is None:
        return make_response(jsonify({
            'Error': 'User not found.'
        }), 404)

    if user.get('role') == 'admin':
        admin_count = users.count_documents({
            'role': 'admin'
        })

        if admin_count <= 1:
            return make_response(jsonify({
                'Error': 'The last administrator account cannot be deleted.'
            }), 400)
```

```
    result = users.delete_one({
        'user_id': user_id
    })

    if result.deleted_count == 0:
        return make_response(jsonify({
            'Error': 'User could not be deleted.'
        }), 500)

    return make_response(jsonify({
        'Message': 'User deleted successfully.'
    }), 200)

#route for admin dashboard

@admin_bp.route('/api/v1.0/admin/dashboard', methods=['GET'])
@jwt_required
@admin_required
def get_admin_dashboard():
    dashboard = {
        'total_users': users.count_documents({}),
```

```
    'total_employees': employees.count_documents({}),

    'total_departments': departments.count_documents({}),

    'total_roles': roles.count_documents({}),

    'total_attendance_records': attendance.count_documents({}),

    'total_leave_requests': leave_requests.count_documents({}),

    'pending_leave_requests': leave_requests.count_documents({
        'status': 'Pending'
    }),

    'active_employees': employees.count_documents({
        'status': 'Active'
    })
}

return make_response(jsonify(dashboard), 200)
```

analytics.py

```
from flask import Blueprint, request, jsonify, make_response

from datetime import datetime

from database import (
    employees,
    departments,
    attendance,
    leave_requests
)

from decorators import jwt_required, admin_required

analytics_bp = Blueprint('analytics', __name__)

#route for employee analytics

@analytics_bp.route('/api/v1.0/analytics/employee-count',
methods=['GET'])
@jwt_required
@admin_required
```

```
def get_employee_count():
    total_employees = employees.count_documents({})

    active_employees = employees.count_documents({
        'status': 'Active'
    })

    inactive_employees = employees.count_documents({
        'status': 'Inactive'
    })

    return make_response(jsonify({
        'total_employees': total_employees,
        'active_employees': active_employees,
        'inactive_employees': inactive_employees
    }), 200)

#route for employees in department

@analytics_bp.route('/api/v1.0/analytics/department-summary',
methods=['GET'])
@jwt_required
@admin_required
def get_department_summary():
```

```
pipeline = [
  {
    '$lookup': {
      'from': 'employees',
      'localField': 'department_id',
      'foreignField': 'department_id',
      'as': 'department_employees'
    }
  },
  {
    '$project': {
      '_id': 0,
      'department_id': 1,
      'name': 1,
      'employee_count': {
        '$size': '$department_employees'
      }
    }
  },
  {
    '$sort': {
      'employee_count': -1
    }
```

```
    }
  ]

  summary = list(departments.aggregate(pipeline))

  return make_response(jsonify(summary), 200)

#route for attendance analytics

@analytics_bp.route('/api/v1.0/analytics/attendance-summary',
methods=['GET'])
@jwt_required
@admin_required
def get_attendance_summary():
  query = {}

  if request.args.get('date'):
    attendance_date = request.args.get('date').strip()

    try:
      datetime.strptime(attendance_date, '%Y-%m-%d')
    except ValueError:
      return make_response(jsonify({
        'Error': 'Date must use YYYY-MM-DD format.'
```

```
        }), 400)

    query['date'] = attendance_date

total_records = attendance.count_documents(query)

pipeline = [
    {
        '$match': query
    },
    {
        '$group': {
            '_id': '$status',
            'count': {'$sum': 1}
        }
    },
    {
        '$sort': {'_id': 1}
    }
]

status_counts = {}
```

```
    for result in attendance.aggregate(pipeline):
        status_counts[result['_id']] = result['count']

    return make_response(jsonify({
        'date': query.get('date', 'All dates'),
        'total_attendance_records': total_records,
        'status_summary': status_counts
    }), 200)

#route for leave request analytics

@analytics_bp.route('/api/v1.0/analytics/leave-summary',
methods=['GET'])
@jwt_required
@admin_required
def get_leave_summary():
    query = {}

    if request.args.get('employee_id'):
        query['employee_id'] = request.args.get('employee_id').strip()

    total_requests = leave_requests.count_documents(query)

    status_pipeline = [
```

```
  {
    '$match': query
  },
  {
    '$group': {
      '_id': '$status',
      'count': {'$sum': 1}
    }
  }
]

status_summary = {}

for result in leave_requests.aggregate(status_pipeline):
  status_summary[result['_id']] = result['count']

type_pipeline = [
  {
    '$match': query
  },
  {
    '$group': {
      '_id': '$leave_type',
```

```
            'count': {'$sum': 1}
        }
    }
]

type_summary = {}

for result in leave_requests.aggregate(type_pipeline):
    type_summary[result['_id']] = result['count']

return make_response(jsonify({
    'employee_id': query.get('employee_id', 'All employees'),
    'total_leave_requests': total_requests,
    'status_summary': status_summary,
    'leave_type_summary': type_summary
}), 200)
```

attendance.py

```
from flask import Blueprint, request, jsonify, make_response

from datetime import datetime
```

```
from database import attendance, employees

from decorators import jwt_required, admin_required

attendance_bp = Blueprint('attendance', __name__)

#route to get all attendances

@attendance_bp.route('/api/v1.0/attendances', methods=['GET'])
@jwt_required
@admin_required
def get_all_attendance():
    query = {}

    if request.args.get('employee_id'):
        query['employee_id'] = request.args.get('employee_id').strip()

    if request.args.get('date'):
        query['date'] = request.args.get('date').strip()

    if request.args.get('status'):
        status = request.args.get('status').strip()
```

```
    if status not in ['Present', 'Absent', 'Late']:
        return make_response(jsonify({
            'Error': 'Status must be Present, Absent or Late'
        }), 400)

    query['status'] = status

  data_to_return = []

  for record in attendance.find(query).sort('date', -1):
    record['_id'] = str(record['_id'])
    data_to_return.append(record)

  return make_response(jsonify(data_to_return), 200)

#route to get one attendance

@attendance_bp.route('/api/v1.0/attendances/<string:attendance_id>',
methods=['GET'])
@jwt_required
@admin_required
def get_one_attendance(attendance_id):
  record = attendance.find_one({
```

```
        'attendance_id': attendance_id
    })

    if record is None:
        return make_response(jsonify({
            'Error': 'Attendance record not found.'
        }), 404)

    record['_id'] = str(record['_id'])

    return make_response(jsonify(record), 200)

#route to add a attendance

@attendance_bp.route('/api/v1.0/attendances', methods=['POST'])
@jwt_required
@admin_required
def add_attendance():
    data = request.form

    if not data:
        return make_response(jsonify({
            'Error': 'No form data was provided.'
```

```
    }), 400)

required_fields = [
    'attendance_id',
    'employee_id',
    'date',
    'status'
]

missing_fields = [
    field for field in required_fields
    if not data.get(field) or not data.get(field).strip()
]

if missing_fields:
    return make_response(jsonify({
        'Error': 'Required fields are missing.',
        'Fields': missing_fields
    }), 400)

attendance_id = data['attendance_id'].strip()
employee_id = data['employee_id'].strip()
attendance_date = data['date'].strip()
```

```
status = data['status'].strip()

check_in = data.get('check_in', '').strip()
check_out = data.get('check_out', '').strip()

try:
  datetime.strptime(attendance_date, '%Y-%m-%d')
except ValueError:
  return make_response(jsonify({
    'Error': 'Date must use YYYY-MM-DD format.'
  }), 400)

if check_in:
  try:
    datetime.strptime(check_in, '%H:%M')
  except ValueError:
    return make_response(jsonify({
      'Error': 'Check-in time must use HH:MM format.'
    }), 400)

if check_out:
  try:
    datetime.strptime(check_out, '%H:%M').time()
```

```
    except ValueError:
        return make_response(jsonify({
            'Error': 'Check-out time must use HH:MM format.'
        }), 400)

if check_out and not check_in:
    return make_response(jsonify({
        'Error': 'Check-in time is required to check-out .'
    }), 400)

if check_in and check_out and check_out <= check_in:
    return make_response(jsonify({
        'Error': 'Check-out time must be later than check-in time.'
    }), 400)

if status not in ['Present', 'Absent', 'Late']:
    return make_response(jsonify({
        'Error': 'Status must be Present, Absent or Late.'
    }), 400)

employee = employees.find_one({
    'employee_id': employee_id
})
```

```
if employee is None:
    return make_response(jsonify({
        'Error': 'Employee not found.'
    }), 404)

if attendance.find_one({'attendance_id': attendance_id}):
    return make_response(jsonify({
        'Error': 'This attendance ID already exists.'
    }), 409)

existing_record = attendance.find_one({
    'employee_id': employee_id,
    'date': attendance_date
})

if existing_record:
    return make_response(jsonify({
        'Error': 'Attendance for this employee on this date already exists.'
    }), 409)

record = {
    'attendance_id': attendance_id,
```

```
        'employee_id': employee_id,
        'date': attendance_date,
        'check_in': check_in or None,
        'check_out': check_out or None,
        'status': status
    }

    attendance.insert_one(record)

    new_record = attendance.find_one({
        'attendance_id': attendance_id
    })

    new_record['_id'] = str(new_record['_id'])

    return make_response(jsonify({
        'Message': 'Attendance record created successfully.',
        'Attendance': new_record
    }), 201)

#route to update an attendance

@attendance_bp.route('/api/v1.0/attendances/<string:attendance_id>',
methods=['PUT'])
```

```
@jwt_required
@admin_required
def update_attendance(attendance_id):
    record = attendance.find_one({
        'attendance_id': attendance_id
    })

    if record is None:
        return make_response(jsonify({
            'Error': 'Attendance record not found.'
        }), 404)

    data = request.form.to_dict()

    if not data:
        return make_response(jsonify({
            'Error': 'No form data was provided.'
        }), 400)

    allowed_fields = [
        'employee_id',
        'date',
        'check_in',
```

```
    'check_out',
    'status'
]

invalid_fields = [
    field for field in data
    if field not in allowed_fields
]

if invalid_fields:
    return make_response(jsonify({
        'Error': 'One or more fields cannot be updated.',
        'Invalid fields': invalid_fields
    }), 400)

for field in data:
    data[field] = data[field].strip()

employee_id = data.get('employee_id', record['employee_id'])

if not employee_id:
    return make_response(jsonify({
        'Error': 'Employee ID cannot be empty.'
```

```
    }), 400)

if 'employee_id' in data:
    employee = employees.find_one({
        'employee_id': employee_id
    })

    if employee is None:
        return make_response(jsonify({
            'Error': 'Employee not found.'
        }), 404)

attendance_date = data.get('date', record['date'])

if not attendance_date:
    return make_response(jsonify({
        'Error': 'Date cannot be empty.'
    }), 400)

try:
    datetime.strptime(attendance_date, '%Y-%m-%d')
except ValueError:
    return make_response(jsonify({
```

```
            'Error': 'Date must use YYYY-MM-DD format.'
        }), 400)

    check_in = data.get('check_in', record.get('check_in'))

    if check_in:
        try:
            datetime.strptime(check_in, '%H:%M')
        except ValueError:
            return make_response(jsonify({
                'Error': 'Check-in time must use HH:MM format.'
            }), 400)

    check_out = data.get('check_out', record.get('check_out'))

    if check_out:
        try:
            datetime.strptime(check_out, '%H:%M')
        except ValueError:
            return make_response(jsonify({
                'Error': 'Check-out time must use HH:MM format.'
            }), 400)
```

```
if check_out and not check_in:
    return make_response(jsonify({
        'Error': 'Check-in time is required when check-out time is provided.'
    }), 400)

if check_in and check_out and check_out <= check_in:
    return make_response(jsonify({
        'Error': 'Check-out time must be later than check-in time.'
    }), 400)

status = data.get('status', record['status'])

if status not in ['Present', 'Absent', 'Late']:
    return make_response(jsonify({
        'Error': 'Status must be Present, Absent or Late.'
    }), 400)

existing_record = attendance.find_one({
    'employee_id': employee_id,
    'date': attendance_date,
    'attendance_id': {'$ne': attendance_id}
})
```

```
    if existing_record:
        return make_response(jsonify({
            'Error': 'Attendance for this employee on this date already exists.'
        }), 409)

    attendance.update_one(
        {'attendance_id': attendance_id},
        {'$set': data}
    )

    updated_record = attendance.find_one({
        'attendance_id': attendance_id
    })

    updated_record['_id'] = str(updated_record['_id'])

    return make_response(jsonify({
        'Message': 'Attendance record updated successfully.',
        'Attendance': updated_record
    }), 200)

#route to delete an attendance
```

```
@attendance_bp.route('/api/v1.0/attendances/<string:attendance_id>',
methods=['DELETE'])
@jwt_required
@admin_required
def delete_attendance(attendance_id):
  record = attendance.find_one({
    'attendance_id': attendance_id
  })

  if record is None:
    return make_response(jsonify({
      'Error': 'Attendance record not found.'
    }), 404)

  result = attendance.delete_one({
    'attendance_id': attendance_id
  })

  if result.deleted_count == 0:
    return make_response(jsonify({
      'Error': 'Attendance record could not be deleted.'
    }), 500)

  return make_response(jsonify({
```

```
        'Message': 'Attendance record deleted successfully.'
    }), 200)
```

auth.py

```
from flask import Blueprint, jsonify, make_response, request
from datetime import datetime, timedelta, timezone
import bcrypt
import jwt

from database import users, blacklist_token, employees
from config import SECRET_KEY
from decorators import jwt_required

auth_bp = Blueprint('auth', __name__)

# Route to register
@auth_bp.route("/api/v1.0/register", methods=["POST"])
def register():

    #Accept JSON or Form Data
    if request.is_json:
```

```
    data = request.get_json()

    if not isinstance(data, dict):
        return make_response(jsonify({
            'Error': 'Invalid JSON data'
        }), 400)

elif request.mimetype in [
    'multipart/form-data',
    'application/x-www-form-urlencoded'
]:

    data = request.form.to_dict()

else:
    return make_response(jsonify({
        'Error': 'Unsupported content type'
    }), 415)

username = data.get('username')
email = data.get('email')
password = data.get('password')
```

```
if not all(isinstance(value, str) for value in
      [username, email, password]):

   return make_response(jsonify({
      'Error': 'All fields must be provided as strings'
   }), 400)

username = username.strip().lower()
email = email.strip().lower()

if not all([username, email, password]):

   return make_response(jsonify({
      'Error': 'All fields are required'
   }), 400)

if len(username) < 3 or len(username) > 30:

   return make_response(jsonify({
      'Error': 'Username must be between 3 and 30 characters'
   }), 400)

if '@' not in email or '.' not in email.split('@')[-1]:
```

```
        return make_response(jsonify({
            'Error': 'Please provide a valid email address'
        }), 400)

    if len(password) < 6 or len(password.encode('utf-8')) > 72:
        return make_response(jsonify({
            'Error': 'Password must be at least 6 characters and not exceed 72
bytes'
        }), 400)



    existing_user = users.find_one({
        '$or': [
            {'username': username},
            {'email': email}
        ]
    })

    if existing_user:

        return make_response(jsonify({
            'Error': 'Username or email already exists'
        }), 409)
```

```
last_user = users.find_one(
    {},
    {'user_id': 1},
    sort=[('user_id', -1)]
)

if last_user:

    last_id = int(last_user['user_id'][3:])
    new_user_id = f'USR{last_id + 1:04d}'

else:
    new_user_id = 'USR0001'

# Hash password using bcrypt
hashed_password = bcrypt.hashpw(
    password.encode('utf-8'),
    bcrypt.gensalt()
).decode('utf-8')

current_time = datetime.now(timezone.utc).isoformat()
```

```
new_user = {

   'user_id': new_user_id,

   'username': username,

   'email': email,

   'password': hashed_password,

   'admin': False,

   'role': 'employee',

   'account_status': 'active',

   'created_at': current_time,

   'updated_at': current_time

}

users.insert_one(new_user)

# Generate JWT token

token = jwt.encode({

'user': username,

'admin': False,

'exp': datetime.now(timezone.utc)
```

```
    + timedelta(minutes=30)
},
SECRET_KEY,
algorithm="HS256")

return make_response(jsonify({

  'Message': 'Registration successful',
  'token': token,
  'user': {
    'user_id': new_user_id,
    'username': username,
    'email': email,
    'admin': False,
    'role': 'employee',
    'account_status': 'active',
    'created_at': current_time,
    'updated_at': current_time
  }

}), 201)
```

```
# Route to login
@auth_bp.route("/api/v1.0/login", methods=["GET"])
def login():

    auth = request.authorization

    if auth:

        user = users.find_one({
            'username': auth.username
        })

        if user is not None:

            if bcrypt.checkpw(
                auth.password.encode('utf-8'),
                user['password'].encode('utf-8')):

                token = jwt.encode({

                    'user': auth.username,
                    'admin': user['admin'],
```

```
            'exp': datetime.now(timezone.utc)
                + timedelta(minutes=30)

        },
        SECRET_KEY,
        algorithm="HS256")

        return make_response(
            jsonify({"token": token}),
            200
        )

    else:
        return make_response(jsonify({
            'Error': 'Wrong Password'
        }), 401)

else:
    return make_response(jsonify({
        'Error': 'Wrong username!'
    }), 401)

return make_response(
```

```
        jsonify({"Error": "could not verify"}),
        401,
        {'WWW-Authenticate': 'Basic realm="login Required"'}
    )

# Route to logout
@auth_bp.route("/api/v1.0/logout", methods=["GET"])
@jwt_required
def logout():

    token = request.headers['x-access-token']

    blacklist_token.insert_one({
        'token': token
    })

    return make_response(jsonify({
        'Message': 'Logout Successful'
    }), 200)
```

departments.py

```
from flask import Blueprint, request, jsonify, make_response
from pymongo.errors import DuplicateKeyError

from database import departments, employees

from decorators import jwt_required, admin_required

department_bp = Blueprint('departments', __name__)

#route to get all departments

@department_bp.route('/api/v1.0/departments', methods= ['GET'])
@jwt_required
@admin_required
def get_all_departments():

    query = {}

    if request.args.get('name'):
        query['name'] = {
            '$regex' :request.args.get('name').strip(),
```

```
        '$options' : 'i'
      }


    data_to_return = []
    for department in departments.find(query):
      department['_id'] = str(department['_id'])

      data_to_return.append(department)

    return make_response(jsonify(data_to_return),200)

#route to get one department

@department_bp.route('/api/v1.0/departments/<string:department_id>',
methods = ['GET'])
@jwt_required
@admin_required
def get_one_department(department_id):

    department = departments.find_one({
      'department_id' : department_id
    })
    if department is not None:
```

```
        department['_id'] = str(department['_id'])

        return make_response(jsonify(department),200)

    return make_response(jsonify({'Error' : 'Department Id is not
valid  !'}),404)

#route to add a department

@department_bp.route('/api/v1.0/departments', methods=["POST"])
@jwt_required
@admin_required
def add_department():

    data = request.form

    if not data:
        return make_response(jsonify({
            'Error': 'No form data was provided.'
        }), 400)

    required_fields = [
        'department_id',
        'name'
```

```
]

missing_fields = [
    field for field in required_fields
    if not data.get(field) or not data.get(field).strip()
]

if missing_fields:
    return make_response(jsonify({
        'Error': 'Required fields are missing.',
        'Fields': missing_fields
    }), 400)

department_id = data['department_id'].strip()
name = data['name'].strip()
description = data.get('description', '').strip()
manager_id = data.get('manager_id', '').strip()

existing_department = departments.find_one({
    'department_id': department_id
})

if existing_department:
```

```
    return make_response(jsonify({
        'Error': 'This department ID is already registered.'
    }), 409)

existing_name = departments.find_one({
    'name': name
})

if existing_name:
    return make_response(jsonify({
        'Error': 'This department name already exists.'
    }), 409)

if manager_id:
    manager = employees.find_one({
        'employee_id': manager_id
    })

    if manager is None:
        return make_response(jsonify({
            'Error': 'The specified manager does not exist.'
        }), 400)
```

```
department = {
   'department_id': department_id,
   'name': name,
   'description': description,
   'manager_id': manager_id if manager_id else None
}

try:
   departments.insert_one(department)

except DuplicateKeyError:
   return make_response(jsonify({
      'Error': 'A department with these unique details already exists.'
   }), 409)
new_department = departments.find_one({
   'department_id': department_id
})
new_department['_id'] = str(new_department['_id'])
return make_response(jsonify({
   'Message': 'Department created successfully.',
   'Department': new_department
}), 201)
```

```
#route to update a department


@department_bp.route('/api/v1.0/departments/<string:department_id>',
methods = ['PUT'])
@jwt_required
@admin_required
def update_department(department_id):


  department = departments.find_one({
    'department_id' : department_id
  })
  if department is None:
    return make_response(jsonify({
      'Error' : 'No department found !'
    }),404)


  data = request.form


  if not data:
    return make_response(jsonify({
      'Error' : 'No form data was provided !'
    }), 400)


  required_field = [
```

```
    'manager_id'
  ]
  missing_field = [
    field for field in required_field if not data.get(field) or not
data.get(field).strip()
  ]

  if missing_field:
    return make_response(jsonify({
      'Error' : 'Required field value missing !',
      'Field' : missing_field
    }),400)

  departments.update_one(
    {'department_id' : department_id},
    { '$set' :{
      'manager_id' : data['manager_id'].strip()
      }
    }
  )

  updated_department = departments.find_one({
    'department_id' : department_id
  })
```

```
    updated_department['_id'] = str(updated_department['_id'])
    return make_response(jsonify({
        'Message':'Department updated successfully ',
        'Department':updated_department
    }),200)


#route to delete a department


@department_bp.route('/api/v1.0/departments/<string:department_id>',
methods = ['DELETE'])
@jwt_required
@admin_required
def delete_department(department_id):

    department = departments.find_one({
        'department_id': department_id
    })
    if department is None:
        return make_response(jsonify({'Error' : 'Department could not
found'}),404)

    department['_id'] = str(department['_id'])
```

```
    result = departments.delete_one({
        'department_id' : department_id
    })

    if result.deleted_count == 0:
        return make_response(jsonify({
            'Error': 'Department could not be deleted !'
        }),500)
    return make_response(jsonify({
        'Message': 'Department deleted successfully'
    }),200)
```

employees.py

```
from flask import Blueprint, jsonify, make_response, request

from datetime import date

from pymongo.errors import DuplicateKeyError
```

```
from database import employees, departments, roles, attendance,
leave_requests


from decorators import jwt_required, admin_required


employee_bp = Blueprint('employee', __name__)



#route to get a employee complete profile


@employee_bp.route('/api/v1.0/employees/<string:employee_id>/profile',
methods=['GET'])
@jwt_required
@admin_required
def get_employee_profile(employee_id):

  def convert_id(document):
    if document is not None and "_id" in document:
      document["_id"] = str(document["_id"])
      return document

  employee = employees.find_one({
    'employee_id': employee_id
  })
```

```
if employee is None:
    return make_response(jsonify({
        'error': 'Employee not found'
    }), 404)

department = departments.find_one({
    'department_id': employee.get('department_id')
})

role = roles.find_one({
    'role_id': employee.get('role_id')
})

attendance_records = list(attendance.find({
    'employee_id': employee_id
}))

leave_records = list(leave_requests.find({
    'employee_id': employee_id
}))
employee = convert_id(employee)
department = convert_id(department)
```

```
    role = convert_id(role)
    attendance_records = [ convert_id(record) for record in
attendance_records]
    leave_records = [ convert_id(record) for record in leave_records]

    return make_response(jsonify({
        'employee': employee,
        'department': department,
        'job_role': role,
        'attendance': attendance_records,
        'leave_requests': leave_records
    }), 200 )

#route to get all employees

@employee_bp.route('/api/v1.0/employees', methods =["GET"])
@jwt_required
@admin_required
def get_all_employees():

    page_num = request.args.get('pn', default=1, type=int)
    page_size = request.args.get('ps', default=10, type=int)

    if page_num is None or page_num < 1:
```

```
    return make_response(jsonify({
        'Error': 'Page number must be a positive integer.'
    }), 400)

if page_size is None or page_size < 1:
    return make_response(jsonify({
        'Error': 'Page size must be a positive integer.'
    }), 400)

if page_size > 50:
    return make_response(jsonify({
        'Error': 'Page size cannot exceed 50.'
    }), 400)

page_start = page_size * (page_num - 1)

query = {}

if request.args.get('department_id'):
    query['department_id'] = request.args.get('department_id').strip()

if request.args.get('role_id'):
    query['role_id'] = request.args.get('role_id').strip()
```

```
    if request.args.get('status'):
        status = request.args.get('status').strip()

        if status not in ['Active', 'Inactive']:
            return make_response(jsonify({
                'Error': 'Status must be Active or Inactive.'
            }), 400)

        query['status'] = status

    if request.args.get('first_name'):
        query['first_name'] = {
            '$regex': request.args.get('first_name').strip(),
            '$options': 'i'
        }

    data_to_return = []

    for employee in employees.find(query).sort('employee_id',
1).skip(page_start).limit(page_size):

        employee['_id'] = str(employee['_id'])
        data_to_return.append(employee)
```

```
    return make_response(jsonify(data_to_return), 200)


#route to get one employee


@employee_bp.route('/api/v1.0/employees/<string:employee_id>',
methods= ["GET"])
@jwt_required
def get_one_employee(employee_id):


    employee = employees.find_one(
        {"employee_id": employee_id},
        {"_id": 0}
    )


    if employee is not None:
        return make_response(jsonify(employee),200)


    else:
        return make_response(jsonify({'Error': 'Invalid Employee ID !'}), 404)


#route to add a new employee
```

```
@employee_bp.route('/api/v1.0/employees', methods =["POST"])
@jwt_required
@admin_required
def add_employee():

  data = request.form

  required_fields = [
    'employee_id',
    'first_name',
    'last_name',
    'email',
    'hire_date',
    'department_id',
    'role_id'
  ]

  missing_fields = [
    field for field in required_fields
    if not data.get(field) or not data.get(field).strip()
  ]

  if missing_fields:
```

```
    return make_response(jsonify({
        'Error': 'Required fields are missing !',
        'fields': missing_fields
    }), 400)

email = data['email'].strip()

# Different validations
if '@' not in email or '.' not in email.rsplit('@', 1)[-1]:
    return make_response(jsonify({
        'Error': 'Please provide a valid email address.'
    }), 400)

try:
    hire_date = date.fromisoformat(data['hire_date'].strip())
except ValueError:
    return make_response(jsonify({
        'Error': 'INVALID date, date must use YYYY-MM-DD format.'
    }), 400)

department = departments.find_one({
    'department_id': data['department_id'].strip()
})
```

```
if department is None:
    return make_response(jsonify({
        'Error': 'The specified department does not exist.'
    }), 400)

role = roles.find_one({
    'role_id': data['role_id'].strip()
})

if role is None:
    return make_response(jsonify({
        'Error': 'The specified job role does not exist.'
    }), 400)

existing_employee = employees.find_one({
    '$or': [
        {'employee_id': data['employee_id'].strip()},
        {'email': email}
    ]
})

if existing_employee:
```

```
    return make_response(jsonify({
        'Error': 'The employee ID or email is already registered.'
    }), 409)

employee = {
    'employee_id': data['employee_id'].strip(),
    'first_name': data['first_name'].strip(),
    'last_name': data['last_name'].strip(),
    'email': email,
    'phone': data.get('phone') or None,
    'date_of_birth': data.get('date_of_birth') or None,
    'hire_date': hire_date.isoformat(),
    'department_id': data['department_id'].strip(),
    'role_id': data['role_id'].strip(),
    'status': 'Active',
    'employment_type': data.get('employment_type', "Permanent")
}

try:
    employees.insert_one(employee)
except DuplicateKeyError:
    return make_response(jsonify({
        'Error': 'An employee with these unique details already exists.'
```

```
    }), 409)

  new_employee = employees.find_one({
    'employee_id' : data['employee_id']
  })
  new_employee['_id'] = str(new_employee['_id'])

  return make_response(jsonify({
    'message': 'Employee created successfully.',
    'data': new_employee
  }), 201)

#route to update  employee

@employee_bp.route('/api/v1.0/employees/<string:employee_id>',
methods=['PUT'])
@jwt_required
def update_employee(employee_id):

  employee = employees.find_one({
    'employee_id': employee_id
  })

  if employee is None:
```

```
    return make_response(jsonify({
        'Error': 'Employee not found.'
    }), 404)

data = request.form.to_dict()

if not data:
    return make_response(jsonify({
        'Error': 'No data was provided.'
    }), 400)

allowed_fields = [
    'first_name',
    'last_name',
    'email',
    'phone',
    'date_of_birth',
    'street',
    'city',
    'postcode',
    'country'

]
```

```
invalid_fields = [
    field for field in data
    if field not in allowed_fields
]

if invalid_fields:
    return make_response(jsonify({
        'Error': 'One or more fields cannot be updated.',
        'Invalid fields': invalid_fields
    }), 400)

for field, value in data.items():
    data[field] = value.strip()

    if not data[field]:
        return make_response(jsonify({
            'Error': f'{field} must not be empty.'
        }), 400)

if 'email' in data:
    email = data['email']
```

```
    if '@' not in email or '.' not in email.rsplit('@', 1)[-1]:
        return make_response(jsonify({
            'Error': 'Please provide a valid email address.'
        }), 400)

    existing_email = employees.find_one({
        'email': email,
        'employee_id': {'$ne': employee_id}
    })

    if existing_email:
        return make_response(jsonify({
            'Error': 'This email address is already registered.'
        }), 409)


if data.get('date_of_birth') is not None:
    try:
        date.fromisoformat(data['date_of_birth'])
    except ValueError:
        return make_response(jsonify({
            'Error': 'Date of birth must use YYYY-MM-DD format.'
        }), 400)
```

```
    try:
        employees.update_one(
            {'employee_id': employee_id},
            {'$set': data}
        )

    except DuplicateKeyError:
        return make_response(jsonify({
            'Error': 'The update conflicts with existing employee information.'
        }), 409)

    updated_employee = employees.find_one(
        {'employee_id': employee_id},
        {'_id': 0}
    )

    return make_response(jsonify({
        'Message': 'Employee information updated successfully.',
        'Employee': updated_employee
    }), 200)

#route to delete an employee
```

```
@employee_bp.route('/api/v1.0/employees/<employee_id>',
methods=['DELETE'])
@jwt_required
@admin_required
def delete_employee(employee_id):

  employee = employees.find_one({
    'employee_id': employee_id
  })

  if employee is None:
    return make_response(jsonify({
      'Error': 'Employee not found.'
    }), 404)

  result = employees.delete_one({
    'employee_id': employee_id
  })

  if result.deleted_count == 0:
    return make_response(jsonify({
      'Error': 'Employee could not be deleted.'
    }), 500)
```

```
    return make_response(jsonify({
        'Message': 'Employee deleted successfully.'
    }), 200)
```

leaves.py

```
from flask import Blueprint, request, jsonify, make_response
from pymongo.errors import DuplicateKeyError
from datetime import datetime

from database import leave_requests, employees
from decorators import jwt_required, admin_required

leave_bp = Blueprint('leaves', __name__)

#route to get all leave requests

@leave_bp.route('/api/v1.0/leaves', methods=['GET'])
@jwt_required
@admin_required
def get_all_leaves():
    query = {}
```

```
if request.args.get('employee_id'):
    query['employee_id'] = request.args.get('employee_id').strip()

if request.args.get('leave_type'):
    leave_type = request.args.get('leave_type').strip()

    if leave_type not in [
        'Annual Leave',
        'Sick Leave',
        'Personal Leave',
        'Unpaid Leave'
    ]:
        return make_response(jsonify({
            'Error': 'Invalid leave type.'
        }), 400)

    query['leave_type'] = leave_type

if request.args.get('status'):
    status = request.args.get('status').strip()

    if status not in ['Pending', 'Approved', 'Rejected']:
        return make_response(jsonify({
```

```
            'Error': 'Status must be Pending, Approved or Rejected.'
        }), 400)

    query['status'] = status

if request.args.get('start_date'):
    start_date = request.args.get('start_date').strip()

    try:
        datetime.strptime(start_date, '%Y-%m-%d')
    except ValueError:
        return make_response(jsonify({
            'Error': 'Start date must use YYYY-MM-DD format.'
        }), 400)

    query['start_date'] = start_date

data_to_return = []

for leave in leave_requests.find(query).sort('requested_at', -1):
    leave['_id'] = str(leave['_id'])
    data_to_return.append(leave)
```

```
    return make_response(jsonify(data_to_return), 200)

#route to get one leave request

@leave_bp.route('/api/v1.0/leaves/<string:leave_id>', methods=['GET'])
@jwt_required
@admin_required
def get_one_leave(leave_id):
    leave = leave_requests.find_one({
        'leave_id': leave_id
    })

    if leave is None:
        return make_response(jsonify({
            'Error': 'Leave request not found.'
        }), 404)

    leave['_id'] = str(leave['_id'])

    return make_response(jsonify(leave), 200)

#route to add a leave request
```

```
@leave_bp.route('/api/v1.0/leaves', methods=['POST'])
@jwt_required
def add_leave():
    data = request.form

    if not data:
        return make_response(jsonify({
            'Error': 'No form data was provided.'
        }), 400)

    required_fields = [
        'leave_id',
        'employee_id',
        'leave_type',
        'start_date',
        'end_date',
        'reason'
    ]

    missing_fields = [
        field for field in required_fields
        if not data.get(field) or not data.get(field).strip()
    ]
```

```
if missing_fields:
    return make_response(jsonify({
        'Error': 'Required fields are missing.',
        'Fields': missing_fields
    }), 400)

leave_id = data['leave_id'].strip()
employee_id = data['employee_id'].strip()
leave_type = data['leave_type'].strip()
start_date = data['start_date'].strip()
end_date = data['end_date'].strip()
reason = data['reason'].strip()

if leave_type not in [
    'Annual Leave',
    'Sick Leave',
    'Personal Leave',
    'Unpaid Leave'
]:
    return make_response(jsonify({
        'Error': 'Invalid leave type.'
    }), 400)
```

```
try:
   datetime.strptime(start_date, '%Y-%m-%d')
except ValueError:
   return make_response(jsonify({
      'Error': 'Start date must use YYYY-MM-DD format.'
   }), 400)

try:
   datetime.strptime(end_date, '%Y-%m-%d')
except ValueError:
   return make_response(jsonify({
      'Error': 'End date must use YYYY-MM-DD format.'
   }), 400)

if end_date <= start_date:
   return make_response(jsonify({
      'Error': 'End date cannot be earlier than start date.'
   }), 400)

employee = employees.find_one({
   'employee_id': employee_id
})
```

```
if employee is None:
    return make_response(jsonify({
        'Error': 'Employee not found.'
    }), 404)

if leave_requests.find_one({'leave_id': leave_id}):
    return make_response(jsonify({
        'Error': 'This leave ID already exists.'
    }), 409)

leave = {
    'leave_id': leave_id,
    'employee_id': employee_id,
    'leave_type': leave_type,
    'start_date': start_date,
    'end_date': end_date,
    'reason': reason,
    'status': 'Pending',
    'requested_at': datetime.now().isoformat(timespec='seconds')
}

try:
```

```
        leave_requests.insert_one(leave)
    except DuplicateKeyError:
        return make_response(jsonify({
            'Error': 'This leave ID already exists.'
        }), 409)

    new_leave = leave_requests.find_one({
        'leave_id': leave_id
    })

    new_leave['_id'] = str(new_leave['_id'])

    return make_response(jsonify({
        'Message': 'Leave request submitted successfully.',
        'Leave': new_leave
    }), 201)

#route to update a leave request

@leave_bp.route('/api/v1.0/leaves/<string:leave_id>', methods=['PUT'])
@jwt_required
def update_leave(leave_id):
    leave = leave_requests.find_one({
```

```
    'leave_id': leave_id
})

if leave is None:
    return make_response(jsonify({
        'Error': 'Leave request not found.'
    }), 404)

data = request.form.to_dict()

if not data:
    return make_response(jsonify({
        'Error': 'No form data was provided.'
    }), 400)

allowed_fields = [
    'employee_id',
    'leave_type',
    'start_date',
    'end_date',
    'reason',
    'status'
]
```

```
invalid_fields = [
    field for field in data
    if field not in allowed_fields
]

if invalid_fields:
    return make_response(jsonify({
        'Error': 'One or more fields cannot be updated.',
        'Invalid fields': invalid_fields
    }), 400)

for field in data:
    data[field] = data[field].strip()

    if not data[field]:
        return make_response(jsonify({
            'Error': f'{field} cannot be empty.'
        }), 400)

employee_id = data.get('employee_id', leave['employee_id'])

if 'employee_id' in data:
```

```
    employee = employees.find_one({
        'employee_id': employee_id
    })

    if employee is None:
        return make_response(jsonify({
            'Error': 'Employee not found.'
        }), 404)

leave_type = data.get('leave_type', leave['leave_type'])

if leave_type not in [
    'Annual Leave',
    'Sick Leave',
    'Personal Leave',
    'Unpaid Leave'
]:
    return make_response(jsonify({
        'Error': 'Invalid leave type.'
    }), 400)

status = data.get('status', leave['status'])
```

```
if status not in ['Pending', 'Approved', 'Rejected']:
    return make_response(jsonify({
        'Error': 'Status must be Pending, Approved or Rejected.'
    }), 400)

start_date = data.get('start_date', leave['start_date'])

try:
    datetime.strptime(start_date, '%Y-%m-%d')
except ValueError:
    return make_response(jsonify({
        'Error': 'Start date must use YYYY-MM-DD format.'
    }), 400)

end_date = data.get('end_date', leave['end_date'])

try:
    datetime.strptime(end_date, '%Y-%m-%d')
except ValueError:
    return make_response(jsonify({
        'Error': 'End date must use YYYY-MM-DD format.'
    }), 400)
```

```
    if end_date < start_date:
        return make_response(jsonify({
            'Error': 'End date cannot be earlier than start date.'
        }), 400)

    leave_requests.update_one(
        {'leave_id': leave_id},
        {'$set': data}
    )

    updated_leave = leave_requests.find_one({
        'leave_id': leave_id
    })

    updated_leave['_id'] = str(updated_leave['_id'])

    return make_response(jsonify({
        'Message': 'Leave request updated successfully.',
        'Leave': updated_leave
    }), 200)

#route to delete a leave request
```

```
@leave_bp.route('/api/v1.0/leaves/<string:leave_id>', methods=['DELETE'])
@jwt_required
def delete_leave(leave_id):
    leave = leave_requests.find_one({
        'leave_id': leave_id
    })

    if leave is None:
        return make_response(jsonify({
            'Error': 'Leave request not found.'
        }), 404)

    result = leave_requests.delete_one({
        'leave_id': leave_id
    })

    if result.deleted_count == 0:
        return make_response(jsonify({
            'Error': 'Leave request could not be deleted.'
        }), 500)

    return make_response(jsonify({
        'Message': 'Leave request deleted successfully.'
```

```
    }), 200)
```

roles.py

```
from flask import Blueprint, request, jsonify, make_response
from pymongo.errors import DuplicateKeyError

from database import roles

from decorators import jwt_required, admin_required

role_bp = Blueprint('roles', __name__)

# Route to get all roles

@role_bp.route('/api/v1.0/roles', methods=['GET'])
@jwt_required
@admin_required
def get_all_roles():

    query = {}
```

```
    if request.args.get('name'):
        query['name'] = {
            '$regex': request.args.get('name').strip(),
            '$options': 'i'
        }

    data_to_return = []

    for role in roles.find(query).sort('role_id', 1):
        role['_id'] = str(role['_id'])
        data_to_return.append(role)

    return make_response(jsonify(data_to_return), 200)

# Route to get one role

@role_bp.route('/api/v1.0/roles/<string:role_id>', methods=['GET'])
@jwt_required
@admin_required
def get_one_role(role_id):

    role = roles.find_one({
        'role_id': role_id
```

```
    })

    if role is None:
        return make_response(jsonify({
            'Error': 'Role not found !'
        }), 404)

    role['_id'] = str(role['_id'])

    return make_response(jsonify(role), 200)

# Route to add a role

@role_bp.route('/api/v1.0/roles', methods=['POST'])
@jwt_required
@admin_required
def add_role():

    data = request.form

    if not data:
        return make_response(jsonify({
            'Error': 'No form data was provided.'
```

```
    }), 400)

required_fields = [
    'role_id',
    'name',
    'min_salary',
    'max_salary'
]

missing_fields = [
    field for field in required_fields
    if not data.get(field) or not data.get(field).strip()
]

if missing_fields:
    return make_response(jsonify({
        'Error': 'Required fields are missing.',
        'Fields': missing_fields
    }), 400)

role_id = data['role_id'].strip()
name = data['name'].strip()
description = data.get('description', '').strip()
```

```
try:
    min_salary = float(data['min_salary'])
    max_salary = float(data['max_salary'])

except ValueError:
    return make_response(jsonify({
        'Error': 'Salary values must be numeric.'
    }), 400)

if min_salary < 0 or max_salary < 0:
    return make_response(jsonify({
        'Error': 'Salary values cannot be negative.'
    }), 400)

if min_salary > max_salary:
    return make_response(jsonify({
        'Error': 'Minimum salary cannot exceed maximum salary.'
    }), 400)

existing_role = roles.find_one({
    'role_id': role_id
})
```

```
if existing_role:
    return make_response(jsonify({
        'Error': 'This role ID is already registered.'
    }), 409)

existing_name = roles.find_one({
    'name': name
})

if existing_name:
    return make_response(jsonify({
        'Error': 'This role name already exists.'
    }), 409)

role = {
    'role_id': role_id,
    'name': name,
    'description': description,
    'min_salary': min_salary,
    'max_salary': max_salary
}
```

```
    try:
        roles.insert_one(role)

    except DuplicateKeyError:
        return make_response(jsonify({
            'Error': 'A role with these unique details already exists.'
        }), 409)

    new_role = roles.find_one({
        'role_id': role_id
    })

    new_role['_id'] = str(new_role['_id'])

    return make_response(jsonify({
        'Message': 'Role created successfully.',
        'Role': new_role
    }), 201)

# Route to update a role

@role_bp.route('/api/v1.0/roles/<string:role_id>', methods=['PUT'])
@jwt_required
```

```
@admin_required
def update_role(role_id):

    role = roles.find_one({
        'role_id': role_id
    })

    if role is None:
        return make_response(jsonify({
            'Error': 'Role not found.'
        }), 404)

    data = request.form.to_dict()

    if not data:
        return make_response(jsonify({
            'Error': 'No form data was provided.'
        }), 400)

    allowed_fields = [
        'name',
        'description',
        'min_salary',
```

```
    'max_salary'
]

invalid_fields = [
    field for field in data
    if field not in allowed_fields
]

if invalid_fields:
    return make_response(jsonify({
        'Error': 'One or more fields cannot be updated.',
        'Invalid fields': invalid_fields
    }), 400)

for field, value in data.items():

    data[field] = value.strip()

    if not data[field]:
        return make_response(jsonify({
            'Error': f'{field} must not be empty.'
        }), 400)
```

```
if 'name' in data:

    existing_name = roles.find_one({
        'name': data['name'],
        'role_id': {'$ne': role_id}
    })

    if existing_name:
        return make_response(jsonify({
            'Error': 'This role name already exists.'
        }), 409)

for field in ['min_salary', 'max_salary']:

    if field in data:
        try:
            data[field] = float(data[field])

        except ValueError:
            return make_response(jsonify({
                'Error': f'{field} must be numeric.'
            }), 400)
```

```
        if data[field] < 0:
            return make_response(jsonify({
                'Error': 'Salary values cannot be negative.'
            }), 400)

min_salary = data.get('min_salary', role['min_salary'])
max_salary = data.get('max_salary', role['max_salary'])

if min_salary > max_salary:
    return make_response(jsonify({
        'Error': 'Minimum salary cannot exceed maximum salary.'
    }), 400)

try:
    roles.update_one(
        {'role_id': role_id},
        {'$set': data}
    )

except DuplicateKeyError:
    return make_response(jsonify({
        'Error': 'The update conflicts with existing role information.'
    }), 409)
```

```
    updated_role = roles.find_one({
        'role_id': role_id
    })

    updated_role['_id'] = str(updated_role['_id'])

    return make_response(jsonify({
        'Message': 'Role updated successfully.',
        'Role': updated_role
    }), 200)

# Route to delete a role

@role_bp.route('/api/v1.0/roles/<string:role_id>', methods=['DELETE'])
@jwt_required
@admin_required
def delete_role(role_id):

    role = roles.find_one({
        'role_id': role_id
    })
```

```
if role is None:
    return make_response(jsonify({
        'Error': 'Role not found.'
    }), 404)

result = roles.delete_one({
    'role_id': role_id
})

if result.deleted_count == 0:
    return make_response(jsonify({
        'Error': 'Role could not be deleted.'
    }), 500)

return make_response(jsonify({
    'Message': 'Role deleted successfully.'
}), 200)
```